

Table of Contents

Table of Contents

Table of Contents

Table of Contents

NAS-AI

Offline AI Knowledge Retrieval System — Technical Architecture, Methodology & Operations Manual

Field

- Version
- Platform
- LLM model
- Embeddings
- Active Collections
- Tailscale IP
- Classification

Change log (v10): Reranker evidence arc — ingestion-time ‘about’ scan (one LLM call per doc-shaped entry stores {about, keywords, related} in chunk payloads; background task with on-demand button, post-ingest switch, per-collection about_scan mode; about_vectors table of embedded about lines); a falsification campaign over subject-guard and about-similarity promotion variants (each fixed one exhibit and broke another — verdicts recorded in code); the SHIPPED result: BGE cross-encoder VETO on the LLM reranker (consensus rule: fires only when the LLM’s chosen doc is lexically invisible to BGE AND a doc in the LLM’s own top-3 carries true-winner evidence; config bge_veto); domain GLOSSARY (config-declared site vocabulary — CTM, Moore=MCM, ‘bad dates’ — injected into rerank prompts when the term appears in the question, synonym OR-expansion in discovery filters, System Config manager); about-closest fallback (anchored semantic closest-match listing when descriptor claims zero, honest weak label); source-anchored feedback (votes bind to the ANSWERING ENTRY — a on the wrong doc stops penalizing the collection once the pipeline answers from the right one); metadata spec cache (per question+collection, TTL, deep-copied); determinism hunt (ORDER BY pinned on sibling/sampling queries; token- level title matching in arbitration; honest ‘No direct match’ listing header); not_equals filter op with negation-aware alias

injection; HALO STRUCTURING — per-action sections (issue/responses/internal_notes/closure), LLM solution extraction at serialization (quoted, source-audited 'solution' chunk; closure is sign-off, not the fix), synthetic status/merges chunks, merge + parent/child mining with family auto-follow fetch, deterministic ticket summary table (click-to-expand sections), Halo API UI panel (pull by ticket / latest N / incremental / ALL as background tasks), write-if-changed fetcher with volatile-field diagnostics; 3B front-of-pipe verdict from runtime counters: 0.4% escalation over 516 messages — KEPT.

Change log (v9): SPEED-01 delivered end to end — merged fast-model front-of-pipe call (chat-vs-retrieval + follow-up rewrite in one 3B call, fully tripwired), per-question intent memoization, batched vocabulary and anchor round-trips, in-process schema cache, capped spec/intent reasons, per-call LLM timing: chat avg 137s -> ~24s, max 1013s -> ~68s, full eval 130+ min -> ~32 min; Memory & Learning — persistent chat sessions, memory-as-a-collection capture (trigger phrases, bare statements with a whole-message interrogative veto, Remember-this button), dedicated 14B fact resolver with a token-preservation gate, auto-confirmed cross-links to mentioned records, question-anchored "From memory" verbatim compose with a relevance filter and a named-field conflict alert at source level, Memories management tab; feedback learning v1 — thumbs capture with undo and a feedback prior as the arbitration tie-break rung; Halo ITSM connector — OAuth2 fetcher (tenant-aware, /api/Status name resolution, sync-time image download against expiring JWTs), combined-JSON ticket files, per-item parser/serializer (header + human actions with who/type/datetime payloads), the first DECLARED schema, config value_aliases with injected-filter trust and the injected-anchor retreat rule, boilerplate prefix/truncate stripping; RELATED-01 (the >=0.80 "too relevant to display" bucket rejoins the ranked list, 'confirmed' ranked first-class, concepts capped with a q-sim measurement harness -> EMBED-01); routing hardening (empty narrow routes widen once to top centroid candidates; 2-char numeric tokens reach anchor tiers gated by namespace-term context); remote operation (local PostgreSQL runbook, media_path_map for mirrored assets, git worktrees, NASAI_PORT).

Change log (v8): multi-item chat queries (CODE-023: deterministic gate + grounded LLM splitter + parallel fan-out with a single_item router override); reference-identifier/alias searchability end-to-end (payload lists + link_keys + nlp_text "References:" line + get_by_identifier either-name fallback — a record is findable by ANY of its exact names); schema-inference overhaul — LLM-primary for tables with heuristic fallback only, junk schemas never persisted (loud failure instead), constrain-don't-hide filename pre-pass (broadened pattern, roles limited to name-class, cardinality rules, leftmost-key tie-break), batch-level detail-table parent-key + discriminator correction (Enums keyed by Tag with Value as enum_value regardless of LLM variance), richest-twin merged schemas, dedicated schema_model config for a larger inference model; XML finalize buffer reset (stale cross-file state caused full re-finalize per file on re-ingest) and per-file schema-failure isolation; metadata SQL hardening — tautology and degenerate-result guards, record-style listings with companion columns, markdown-safe delivery that states its filters, pipeline bookkeeping keys excluded, description_fields exposed as queryable columns, constant-field exclusion, schema-role grounding in the spec prompt, single-schema role-name matcher, upstream intent-mode coercion and record-vs-chunk count coercion, field-name-valued filter drop, exact token=value filter injection, jsonb-array element listings; chat routing anchor family

(filenames, ALL-CAPS codes, identifier prefixes, collection-name tokens, unique schema column names with a proportionality rule and adjacent-word joining) + Tier 1.5 concept-centroid routing that skips the routing LLM entirely on clear margins; deterministic answer-arbitration ladder (exact-key methods > collection-name hit > equals/contains groundedness > doc-title word hits > routing order); cross-links restored and consumed — wikilink persistence added to the rebuild script, bidirectional link lookups, chat related sections re-enabled ranked confirmed-first and capped, /entry/{chunk_id} full-article pages with inline images, gazetteer NER links live (95 confirmed, first recon↔notes bridge); PP-02 lexical repair (camelCase/snake_case word-splitting into nlp_text, per-collection vocabulary token filtering so typos cannot veto the BM25 leg, OR fallback in standalone BM25); low-coverage answer banner (coverage of question content words vs the answering chunk, threshold in config, banner survives LLM synthesis); automated retrieval eval runner (tests/eval_runner.py, both surfaces, incremental results, latency summary) — three full baselines, latest best-on-record; ingestion reachability + skip/fail-reason warnings; schema editor preserves tags/other roles and gains bulk delete; LLM client max_tokens cap + failure logging.

Change log (v7): structured query planner removed entirely (always dry-run, pure latency); BGE reranker excluded from entity_row (25-candidate demotion bug); Phase 4.6 metadata SQL query path (core/metadata_query.py) answers count/list/group-by intents via grounded LLM spec extraction + parameterized SQL, with concept-vector field selection, filter regrounding, zero-result repair and deterministic sentence answers — chat returns these verbatim (LLM wrapper bypass); schema inference upgraded (cardinality signal, deterministic filename-column pre-pass, entity-name/type and comments/description prompt rules, single canonical identifier/primary_name); astro ingestion made schema-driven (ASTRO-004) with filename-parser offset fix (ASTRO-001) and metadata coalescing file_X→X (ASTRO-005); concept vectors CV-05 (doc_type removed from grouping — tags is a data-driven cascade step at ≥20% coverage) and kb_tags payload key renamed to tags; mention matching removed from the cross-link discoverer (superseding CL-01); 50-question retrieval evaluation framework with failure buckets; KB inactive-row filter verified.

Change log (v6): NiceGUI front-end migration (coexists with Streamlit; reliable inline images via direct DOM control); guarded local text-to-SQL analytics engine for metadata/aggregate questions + SQL Inspector tab + router analytics intent; Chat now implemented with an LLM contextualizer (follow-up detection + standalone-query rewrite replacing bracket injection) and a fixed faithfulness guard; discovery/list results rendered as a table instead of raw JSON; collection lists now include configured-but-not-yet-ingested collections. All UI prompts/logic kept domain-agnostic (no hardcoding).

Change log (v5): embedding-truncation fix via token-aware chunking; LLM schema inference with structured output, generic tags role, cross-role dedup, signal-based column pruning and prose-promotion; content-priority table-type detection; non-lossy NLP text; concept-vector grouping tiers + LLM topic/cluster labels; cross-link precision (min length 8, meaningful-context gate) and one-hop wikilink traversal; default model switched to qwen-14b.

1. System Overview

NAS-AI is a fully offline, local AI knowledge retrieval system. It answers natural language questions about ingested documents — FIX protocol dictionaries, Bloomberg field definitions, KB articles, RECON file mappings, Obsidian notes, PDFs, images, DOCX files, and astrophotography metadata — without any connection to external AI services.

Standing constraint: NAS-AI is fully local. No data leaves the local network. No external AI or cloud model connections, ever.

1.1 Infrastructure Components

Component
Mac M1 Pro
TerraMaster NAS (TNAS-RN)
LM Studio
PostgreSQL / pgvector
Tailscale
Streamlit

1.2 Design Principles

- Fully offline — no data leaves the local network, no external AI or cloud connections
- Domain-agnostic — new collections add rows, not tables or code
- No hardcoding — domain logic lives in config or the database, not in Python; column roles are inferred from content, not column names
- LLM-driven — natural language routing and schema classification handled by the local LLM (qwen-14b), not regex patterns; structured output guarantees valid JSON
- Four-stage retrieval pipeline — intent classification, retrieval, reranking, synthesis are separate concerns
- Token-aware ingestion — no chunk exceeds the embedding window, so no content is silently dropped at embed time
- Shared payload schema — all serializers produce category, folder_path, primary_name_field, type_field enabling cross-collection linking
- Every change covered by smoke tests before and after

2. Retrieval Methodology: The Four-Stage Pipeline

Stage
Stage 0
Stage 1
Stage 2
Stage 3

2.1 Stage 0: LLM Intent Classification

qwen-14b classifies intent mode AND extracts role and target for structured discovery routing. Replaced field_maps.json keyword matching and query_terms.json intent_routing (both retired).

Intent

answer

discovery_list

discovery_count

comparison

Discovery_count / discovery_list questions are dispatched to the Metadata SQL Query Path (§2.6) first; the retrieval-based discovery engine is the fallback when spec extraction or validation fails. Group-by questions (“which broker has the most recon files”) also classify as discovery_list (v7), reusing the same hook. Content-based discovery falls through to the RRF discovery path.

2.2 Stage 1: Hybrid Retrieval via RRF

Signal

BM25 (keyword)

Vector (semantic)

Trigram (fuzzy)

2.3 Stage 2: BGE Cross-Encoder Reranking

Setting

Primary reranker

Applied to

Top-K input / output

2.4 Stage 3: Answer Synthesis

Doc Type

structured (FIX/BBG/RECON)

entity_row (KB)

chunked doc (PDF, DOCX, Obsidian)

image

astro

2.5 Cross-Collection Enrichment (Phase 3.5)

When a query returns a result, NAS-AI enriches the answer with related content from other collections using three mechanisms: confirmed exact cross-links, one-hop wikilink traversal from those links, and concept-vector similarity.

2.5.1 Exact Cross-Links

Pre-built confirmed relationships in the `cross_links` table, built via exact-identifier match and trigram name-similarity with confidence-tiered storage (≥ 0.9 auto-confirmed; 0.3–0.9 pending review; < 0.3 skipped). Mention matching was removed in v7 — short broker/client names (Goldman, Moore, CITI) produced mostly-rejected noise even with the CL-01 length gate; the `_meaningful_context()` guard (CL-02) remains in the code for a possible opt-in reintroduction. True free-text linking awaits NER (CL-03).

2.5.2 Wikilink One-Hop Traversal (CL-04, NEW)

- From each confirmed first-hop cross-link target, the router follows one additional hop of confirmed links in the `cross_links` table.
- Enables short chains such as `gsact.txt` → 4.2 → 4.3.1 across linked notes.
- Second-hop sections are deduped, never loop back to the source, are tagged `match_type=wikilink_hop`, and have confidence dampened $\times 0.9$.
- Payoff scales with how many confirmed wikilinks exist (populated by the related-titles linker / planned NER pass).

2.5.3 Concept Vector Similarity

Semantic cross-collection discovery using HDBSCAN clustering over BGE embeddings. The matched answer chunk's cluster centroid is compared against all other collections' centroids; related clusters surface as expandable “Related Topics” sections. See §5 for how clusters are grouped and labeled.

2.5.4 Ask Tab Controls

- Show exact cross-links — surfaces confirmed exact/name relationships (and their one-hop wikilink targets).
- Show related topics — surfaces concept-vector similarity matches.

2.6 Metadata SQL Query Path (Phase 4.6 — NEW in v7)

Semantic retrieval answers “what does X say” but cannot answer “how many / which / count where” — retrieval-based counting is top-k capped and wrong (4 instead of 178 KB articles; 41 instead of 947 FIX tags). `core/metadata_query.py` (v7) answers `discovery_count` / `discovery_list` intents via direct parameterized SQL against the `chunks` table. The router tries `run_metadata_query()` first; the discovery engine is the fallback. The LLM never writes SQL — it only produces a validated spec. The earlier free-form text-to-SQL analytics engine remains available in the SQL Inspector tab only (it guessed wrong columns when auto-dispatched).

- **Grounded spec extraction** — the collection’s ACTUAL payload keys (sampled 200 chunks) plus real table columns are handed to the LLM together with the distinct values of every low-cardinality (≤ 20) field; qwen returns {operation: count | count_distinct | list_distinct | group_by, target_field, filters[{field, op(equals|contains), value}]}]. Any field not in the real key set, or any malformed filter, rejects the spec and falls back to discovery. Filter values **MUST** be copied from the listed real values.
- **Concept-vector field selection** — for group_by / list_distinct the question is embedded against each field’s concept-vector GROUP LABELS (raw-value embedding failed: job names lexically beat broker names for “which broker...”). The LLM’s target_field is overridden only when it picked a schema-generic field (that collection’s primary_name/identifier per its stored schema) — the guard keeps correct specific picks like date-obs intact. count_distinct defaults to the identifier role (the schema’s unique key).
- **Deterministic repairs** — (1) concept-label filter injection: when the spec has NO filters, the question is matched against concept labels (cosine ≥ 0.6 , margin ≥ 0.05 over the runner-up) and an equals filter is injected (fixed “how many FIX tags” $\rightarrow$ identifier_namespace=tag $\rightarrow$ 947). (2) Filter regrounding: a filter value exactly matching a listed field value is rewritten to equals on that field — the LLM’s own field is checked FIRST, then others in sorted order; unordered iteration here was the root cause of four different answers for one question. (3) Zero-result repair: filters yielding 0 rows are dropped individually, keeping any filter that alone yields rows.
- **Fixed parameterized templates** — COUNT, COUNT DISTINCT, SELECT DISTINCT, GROUP BY + ORDER BY count. SELECT-only, fields validated pre-SQL; the LLM never writes SQL. Answers are deterministic sentences (“There are 947 matching identifier(s).”); list_distinct collapses all-ISO-timestamp value sets to unique dates.
- **Chat bypass** — method == metadata_sql results are returned to the user VERBATIM, skipping the conversational LLM wrapper (which misread bare numbers as tag IDs and produced “lacks context” disclaimers).
- **Verified & known limits** — standalone 5/5: KB articles=178, FIX-mention articles=48, FIX tags=947, prime-broker list via the type field, Goldman group-by $\rightarrow$ Goldman: 16. Open: LLM spec variance across runs (MQ-01), chat fan-out variance (MQ-02); NULL identifiers from merged Excel cells undercount contains-filters (CODE-024). The SQL Inspector tab still hosts the guarded free-form text-to-SQL analytics engine (NL $\rightarrow$ editable SQL $\rightarrow$ run, read-only).

2.7 Chat: Multi-Turn Contextualization (NEW)

The Chat tab (core/chat_engine.py) adds conversational, multi-turn retrieval over the same pipeline. Each turn: classify chat-vs-retrieval intent $\rightarrow$ contextualize $\rightarrow$ route to 1–3 collections $\rightarrow$ retrieve in parallel $\rightarrow$ synthesize a grounded answer with optional related sections.

- **LLM contextualizer** (replaces bracket injection) — contextualize_query() returns {is_followup, standalone_query}. A new/standalone question passes through UNCHANGED; a genuine follow-up (“what about the other one?”) is rewritten into a self-contained query by pulling only the missing subject from recent history. The

classifier defaults to standalone to avoid false-positive follow-ups, and never dumps prior answer text into the query — which was the root cause of “senseless second answers”.

- **History gating** — prior turns are passed to collection routing and answer generation ONLY when the turn is an actual follow-up, so a previous topic can’t pollute a new question.
- **Grounded synthesis + faithfulness guard** — answers are built from retrieved data verbatim; a key-term faithfulness check falls back to the raw retrieved answer if the LLM drifts. (A v5-era NameError that silently disabled this guard is fixed in v6.)
- **Fast rewrite model (optional)** — `local_llm.rewrite_model` / `fast_model` points the rewrite step at a smaller model; defaults to the main model.
- **Two-tier collection routing (v7)** — every collection carries a mandatory `routing_description` (entered at creation), auto-extended with its concept-vector cluster topics for doc-type collections; read live from `collections.json` at query time (no re-ingest needed). Tier 1 explicit-identifier hits are merged AFTER the Tier 2 LLM ordering, never returned early — procedural intent wins over raw record lookups. Empty routing descriptions misroute (astro image questions went to obsidian until descriptions were added).
- **No-answer guard + metadata bypass (v7)** — when the primary answer is “No answer found”, the LLM wrapper is skipped entirely (it fabricated plausible note titles) and a clean “couldn’t find” message is returned; `metadata_sql` results are likewise returned verbatim (§2.6).
- **Foundation for memory/learning (planned)** — the standalone query + `is_followup` flag are the hooks persistent sessions, rolling summaries and a rated-Q&A “fast lane” will build on (§10.3).

2.7 Chat Routing & Answer Arbitration (NEW in v8)

Chat collection selection is now anchor-first and deterministic; the routing LLM survives only as a fallback for genuinely murky questions.

Anchor family — everything the user LITERALLY wrote, verified against the system’s own data (nothing named in code):

Tier

1
1.25
1.2
1.2b
1.5

Tier 1.2b proportionality: a multi-word column needs ≥ 2 of its words in the question (‘prime brokers’ earns ‘Prime Broker’; ‘moore’ alone does NOT earn ‘Moore file name’); single-word columns need ≥ 5 characters (‘Mnemonic’ anchors, bare ‘Name’ does not); adjacent column words join (‘filenames’ covers ‘file’+‘name’). Column names shared by

more than one collection never anchor. Tier 1.5 (config centroid_routing: min_sim 0.6, margin 0.08, max 3): when the leading collection clears the threshold, the Tier 2 routing LLM call is SKIPPED — deterministic selection and one less LLM call per turn. Anchors merge in after centroid ranking and are never dropped by the cap (each takes a distinct slot; anchors never evict each other).

Arbitration ladder — the fan-out’s best answer is chosen by a deterministic score tuple, replacing “first non-empty in routing order”:

1. Exact-key methods (identifier/namespace/enum/relationship lookups) beat everything — a record keyed by the question’s own identifier outranks any semantic document.
2. Collection-name hit (data answers only; muted for documents so “recon file missing, what do I do” keeps its runbook).
3. Graded groundedness for metadata answers: an EQUALS filter whose value matches a question token (identifier_namespace = tag) outranks a contains-filter on free text, which outranks no filter. “Equality is a claim, substring is a shrug.”
4. Doc-vs-doc (kb/obsidian twins): the answer whose TITLE matches more question words wins.
5. Routing order — final tie-break only.

Cross-collection composition (“compose, don’t blend”): runner-up STRUCTURED answers (records/metadata listings — never a second procedure document) are appended verbatim AFTER synthesis as “From {collection}:" blocks. The LLM never sees them — zero contamination, zero step-blending. Relevance gate: a question word must be the PREFIX of a listed record identifier; bare counts never compose. Result shape for cross-collection questions: procedure (synthesized) + records (verbatim) + related links (navigable).

Metadata SQL guards added in v8 (on top of §2.6): tautology guard (list/group target colliding with an equals-filter field retargets to identifier); degenerate-result fallback to the discovery engine; record-style listings with companion columns (identifier — primary name; primary name — truncated description); answers state their filters and render markdown-safe; pipeline bookkeeping keys (doc_type, link_keys, ...) excluded from the LLM’s field menu; description_fields labeled columns are first-class queryable fields; constant fields (one distinct value) excluded; the spec prompt is grounded with schema role meanings (“identifier holds: Moore file name”); upstream intent (list vs count) coerces the spec’s operation; plain COUNT(*) is coerced to count DISTINCT records unless the question literally asks for rows/chunks; filters whose value is a FIELD NAME are dropped (schema/data confusion); a question token exactly equal to a listed field value injects an equals filter deterministically (‘tags’ → identifier_namespace = tag); jsonb-array fields list their ELEMENTS. Never drop all zero-result filters: when every filter matches nothing, that IS the answer (no-answer traps stay honest).

3. Database Schema

PostgreSQL with pgvector is the sole primary store. The schema is generic — domain knowledge lives in the payload JSONB column and the schema JSON stored in PostgreSQL.

3.1 Core Tables

Table

chunks
enum_values
files
schemas
cross_links
concept_vectors

3.2 Key Columns in chunks

Column

collection_name
identifier
primary_name / *_field
type / type_field
category / folder_path
tags (was kb_tags)
chunk_index / chunk_total
chunk_part / chunk_part_total
doc_type
section_heading
nlp_text / nlp_text_tsv
embedding
payload

3.3 cross_links Table

Column

source/target_collection + identifier
match_type
confidence
status

3.4 concept_vectors Table

Column

collection / group_field / group_value
cluster_id
centroid
anchor_chunk_ids / anchor_texts

4. Ingestion Pipeline

4.1 Pipeline Stages

Stage

1. File discovery
2. Change detection
3. Parsing
4. Field filtering
5. Schema mapping
6. Serialization
7. Embedding
8. PostgreSQL write
9. Concept vectors

4.2 Schema Inference (Updated)

Schema inference maps each source column to a role (identifier, primary_name, aliases, description, type, tags, enum_value, enum_name, reference_identifier, other). It is LLM-first with heuristic fallback, and file-agnostic — roles are decided by content, not hardcoded column names. v7 adds a cardinality signal, a deterministic filename-column pre-pass, and content-semantics prompt rules that fixed real misassignments on the RECON mapping file.

- **Cache-first:** a saved schema in the PostgreSQL schemas table is reused (delete it to force re-inference).
- **Heuristic pass** (config/structured_roles.json keyword patterns) runs first and is fast.
- **LLM escalation** (qwen-14b) runs when the heuristic misses identifier/primary_name OR when the table is detected as entity_row (article-style) — where tag detection and free-text roles benefit from the LLM.
- **Structured output:** the LLM call uses response_format json_schema, guaranteeing valid JSON in the exact {role: [columns]} shape (no more parse failures).
- **Generic tags role:** a content-based 'tags' role lets the model identify multi-value keyword columns (e.g. KB kbtags) in any file, surfaced into the kb_tags payload field.
- **Cross-role de-duplication:** structured output guarantees shape but not one-column-one-role, so each column is assigned to exactly one role by priority.

- **Signal-based column pruning:** near-empty columns (e.g. blank date-matrix columns) are dropped before the LLM call so wide-but-real tables still get inference; pruned columns route to 'other'.
- **Wide-file safety net:** if a sheet is still very wide after pruning, inference falls back to the heuristic instead of risking a bad/oversized LLM call.
- **Prose-promotion:** any 'other' column whose values are long free-text is promoted into 'description', so substantive content (e.g. a resolution column) lands in the role answers are built from — regardless of heuristic/LLM quirks.
- **Cardinality signal (v7)** — distinct/non-empty counts per column are computed and passed to the LLM with explicit flags: near-unique (≥ 0.9) → identifier candidate; low-cardinality ($\leq 10\%$) → type/category, never identifier. Multiple near-unique picks collapse to a single canonical identifier (most-unique wins; the rest become aliases); same for primary_name.
- **Deterministic filename pre-pass (v7)** — columns where $\geq 70\%$ of sampled values match a filename pattern (word characters + dot + short alphabetic extension) are auto-classified as reference_identifier BEFORE the LLM call and excluded from the prompt. Three prompt-tuning attempts failed to stop qwen classifying a .bat script column as type; the 5-line regex pre-pass fixed it permanently and generically. Prompt tuning has a ceiling; deterministic pre-passes don't.
- **Content-semantics prompt rules (v7)** — columns whose values are entity/organization names repeating across rows (brokers, clients, vendors) → type; a comments/notes column → description, not primary_name (primary_name is "the column a user would call the record by"); numeric measurements (ra, dec, exposure, gain, temperature) and dates/timestamps → other, even if low-cardinality. On recon_assist_file these moved Prime Broker → type, Tidal Job Name → primary_name, Comment → description.
- **Manual override** remains available via the Validation/Schema tab for edge cases.

Table-type detection (entity_row vs structured) is content-first: a table whose rows carry long free-text (description median ≥ 500 or max $\geq 1,500$ chars) is entity_row, even when it also has a reference/enum column — this check runs BEFORE the reference/enum rule. This is why kb_docs (long resolutions) is entity_row while bbg_fields (one-line field labels) is structured, though both are tables.

4.3 Chunking & the Embedding Window (NEW)

The embedding model (bge-large) silently ignores input beyond ~2,500 chars (~512 tokens) — measured directly: a vector stops changing once a long document's prefix passes ~2,500 chars. Before this fix, roughly one KB article in three exceeded the cap, the worst losing ~78% of its text from the embedding. Ingestion now chunks long content so every vector fits the window.

- **Shared splitter** (core/chunking.py): targets ~1,800 chars/chunk with ~150-char overlap, breaking on paragraph then sentence boundaries.

- **Entity rows (P0):** long records split into multiple chunks that SHARE identifier/primary_name/source_file/link_keys/kb_tags; each adds chunk_index/chunk_total and repeats the title so it embeds self-contained.
- **Doc/PDF (P0b):** oversized block-chunks are capped via the same splitter at the doc and pdf serializer outputs (covers txt, md, rtf, docx, pdf).
- **Result:** 0 chunks over the cap across all collections (kb_docs 33.7% truncated → 0%).
- **Not yet capped:** structured tables and standalone images — their rows/text are short by nature, so no truncation occurs today; revisit if a huge single field ever appears.

4.4 Non-Lossy NLP Text (NEW)

- All field text is HTML-stripped (entities unescaped, tags removed) and included rather than dropped on length or '<' — content is preserved, and length is handled by chunking, not by discarding.
- **Redundant-field dedup:** a value that is a near-duplicate of one already included (e.g. a *Markdown column mirroring its plain-text counterpart) is skipped, keeping the index lean.

4.5 FIX Version Consolidation (Phase 1b — COMPLETE)

- merge_rows_by_version() groups rows by Tag (fields) or (Tag, Value) (enums) across version files at finalize.
- Latest-version-wins for canonical Name/Type/Description/SymbolicName on conflicts; full _version_history retained.
- Verified: Tag 22 returns all 19 enum values (9 from FIX42 + 10 from FIX44).

4.6 Astro Ingestion: Schema-Driven Roles & Metadata Coalescing (NEW in v7)

Astro files (FITS/XISF) previously assigned roles in serializer code — the exact hardcoding pattern removed from tables. v7 makes astro fully schema-driven and fixes two ingestion defects that made metadata queries unstable.

- **Schema-driven roles (ASTRO-004)** — ASTRO/schema_inference_astro.py mirrors the table flow: existing schema in PostgreSQL → LLM inference over metadata keys + sample values → heuristic fallback. The schema is collection-level (source_file_stem = collection_name): one LLM call per collection, reused by every file. The serializer resolves primary_name through the schema's role fields, with file_target appended as the final fallback (covers headers without OBJECT). Verified on M51: 505 chunks, primary_name = "m 51" across all files.
- **Filename parser offset (ASTRO-001)** — the config-driven filename pattern (config/astro_metadata.json) fixed file_target at position 1, assuming a frame-type prefix; M42_2.0s... parsed target = "2.0s". _apply_filename_pattern now carries a positional offset: when a position-0 field's allowed_values don't match, the offset shifts subsequent positional fields back and the field takes its configured default — file_frame_type gained "default": "light" (no frame token = light frame).
- **Metadata coalescing (ASTRO-005)** — duplicate concepts (gain vs file_gain, object vs file_target) made every downstream filter pick a coin-flip: the same gain question

returned 301/276/147/172 across runs. `_merge_filename_metadata` now applies a generic fill — every `file_X` value fills an empty canonical X; `file_*` keys are kept in the payload for traceability but excluded from schema-inference input. Coalesce duplicates at ingest, not at query time.

- **Routing descriptions (ASTRO-006)** — astro collections need `routing_description` terms covering images, targets, gain, rotation, dates; empty descriptions routed image questions to obsidian (read live from `collections.json` — no re-ingest required after editing).

4.7 Supported File Types

Filetype

xml

tables

docs

pdf

image

astro

5. Cross-Collection System (Phase 3.5)

5.1 Architecture Overview

Layer 1: Exact Cross-Links (`cross_links` table)

Built by `core/cross_link_discoverer.py` with three strategies:

- **Exact identifier match** — confidence 1.0 (skips plain short numeric IDs to avoid false positives).
- **Name/primary_name trigram similarity** — confidence = trigram score (>0.3).
- **Mention matching** — source identifier/type appears in target text. Precision hardened: minimum term length raised from 4 to 8 (CL-01), and a meaningful-context gate (CL-02) requires the term to appear as a whole word with enough surrounding words — not a bare list/reference entry. Confidence length-scaled 0.45/0.55/0.70; filtered by `generic_terms`.

Manual review via Collections tab: Confirm / Reject / Reject+Ignore Term (auto-adds to `generic_terms`).

Layer 2: Concept Vector Similarity (`concept_vectors` table)

Built by `core/concept_vector_builder.py`. Grouping is data-driven and tiered, then HDBSCAN clusters within each group, and generic labels are upgraded via the LLM:

- **Grouping cascade (CV-05, v7 — doc_type removed)**: `folder_path` → `section_heading` >1 distinct (CV-01) → `identifier_namespace` >1 → `meaningful type` >1 → `category` →

tags populated on $\geq 20\%$ of chunks $\rightarrow$ source_file fallback. One data-driven cascade for ALL collections; the former `_is_entity_row_collection` doc_type fork is deleted — it would have routed `astro_catalog` (13,336 untagged entity_row chunks) through the tag path and fired ~ 445 Tier-2 LLM calls per rebuild. `doc_type` is a weak proxy for grouping; route on the actual presence of a usable field.

- **Tag grouping (CV-02/03 — any collection reaching the tags step):** Tier 1 — score tags (payload key renamed from `kb_tags` in v7) by inverse document frequency and pick the most distinctive *shared* tag as the group value (no LLM). Tier 2 — for chunks with no/generic tags, compute TF-IDF, take the top terms, and batch-call the LLM for a shared topic label.
- **HDBSCAN clustering** with `min_cluster_size` adaptive to group size; multi-label assignment via $\cosine \geq 0.75$.
- **CV-04** — after clustering, any group value that is still a filename or generic value is relabeled by the LLM from the cluster's anchor texts, using a high running `cluster_id` so relabeled clusters can't collide.
- **CV-03b — label consolidation REVERTED** — an LLM pass to merge synonymous labels (AutoSys / AutoSys Job) over-merged distinct topics on qwen-14b (QA Archive $\rightarrow$ PROD Archive, Repo Collateral $\rightarrow$ Recon Missing Files) even with a guarded prompt. `_llm_consolidate_labels` remains in the file, unwired; revisit with a larger model (Phase 5). Duplicate labels are cosmetic — each keeps its own centroid, so retrieval is unaffected.
- **Stale rows** for a collection are cleared before each rebuild, so old grouping fields / pre-relabel filenames don't linger.
- **Anchor chunks:** top-5 most representative per cluster stored for reference.

Layer 3: Gazetteer NER Entity Extraction (LIVE in v8)

`run_identifier_ner` scans chunk text for known identifiers, aliases and reference filenames from OTHER collections (the searchability overhaul made these matchable) and mints pending-review links. First production run: 95 links (`kb_docs` 16, `obsidian` 79) — the first-ever recon $\rightarrow$ notes bridge, catching both filename AND job-name mentions — sampled clean and confirmed via the Knowledge Graph review. Lookups are BIDIRECTIONAL (links are edges, not arrows): an `obsidian` $\rightarrow$ recon link surfaces when the query answers from recon. This is what unlocked the CL-04 traversal's full effect (`gsact.txt` $\rightarrow$ 4.1 Checking the Tidal Recon Job $\rightarrow$ 4.2 Checking Files on `us1-proc02` in one answer). Wikilink persistence is part of `rebuild_cross_links.py`; background-task interruption (UI restart mid-queue) was the root cause of the links vanishing after re-ingests — BG-01 covers startup re-queueing.

5.2 Query-Time Enrichment Flow

- `identifier_lookup` and fallback paths both fetch confirmed exact cross-links first.
- CL-04: follow one hop of confirmed wikilinks from each first-hop target.
- Both paths: run `find_concept_links()` for semantic concept matches.
- Results merged, deduplicated, RANKED confirmed-first (exact/ner/wikilink $\rightarrow$ hop $\rightarrow$ similarity $\rightarrow$ concept) and capped at 5 (chat) — confirmed procedure notes float above concept noise.

- Every related section carries “Open full article ↗” → `/entry/{chunk_id}`: a standalone page rendering the FULL document (sibling chunks merged — by identifier for KB articles, by source file for notes) with all embedded images inline.
- All rendering is markdown-safe: underscores in identifiers (`019_W_RECON_GOLDMAN_PRIO_PULL`) are escaped outside code spans instead of being eaten as italics.

6. Collections

Name

xml_test
 bbg_fields
 kb_docs
 recon_assist_file
 obsidian
 pdf_test
 image_test
 docx_test
 astro_test
 astro_catalog
 M51

7. User Interface

The UI is migrating from Streamlit to **NiceGUI** (`core/ui_app.py` → `ui/app.py`). Streamlit reran the whole script on every interaction and issued N+1 DB calls over Tailscale, making it slow; NiceGUI controls the DOM directly (which also fixes inline image rendering) and reuses the same UI-agnostic data layer.

7.1 NiceGUI App (primary)

Run with: `python ui/app.py` from the `nas-ai/claude` directory, then open `http://localhost:8080`. Shares core/ logic via `core/ui_data.py` (a framework-agnostic data layer used by both UIs and any future API).

Tab

Collections
 Ask
 Chat
 Ingestion
 Knowledge Graph

SQL Inspector

Debug

Validation

Preview

System Config

Data Prep

Notes: tabs live inside a sticky header so they stay visible while scrolling; discovery/list answers render as a table (not raw JSON); collection dropdowns include configured-but-not-yet-ingested collections (union of collections.json + DB + stored data).

7.2 Streamlit App (legacy, still runnable)

Run with: `streamlit run core/ui_app.py`. Retains the full tab set (Collections, Ingestion, Validation, Ask, Preview, SQL Inspector, System Config, Data Prep). Kept until the NiceGUI tabs reach parity, then retired.

8. Configuration Files

File

`config/system.json`

`config/nlp_config.json`

`config/structured_roles.json`

`config/synonyms.json`

`config/doc_query_hints.json`

`config/filetypes.json`

`config/collections.json`

9. Smoke Tests & Verification

Smoke suites must pass before any commit. Ingestion changes are additionally validated by the diagnostic scripts below (read-only unless noted).

Script

`diag_truncation.py` / `diag_truncation_proof.py`

`diag_schema_guardrails.py`

`diag_verify_ingestion.py`

`diag_verify_concepts.py`

`diag_analytics.py`

`diag_chat_scenarios.py`

smoke_fix / bbg / kb / recon / obsidian

9.9 Automated Retrieval Evaluation (NEW in v8)

tests/eval_runner.py runs all 50 ground-truth questions (NAS_AI_Retrieval_Eval_v1.md) through BOTH surfaces — Ask (run_query_with_method against the expected collection) and Chat (chat_turn, auto-routed) — writing incremental markdown + JSON results with per-question latency and a latency summary (avg/p95/max + slowest five). Verdicts remain human. Category filters (python tests/eval_runner.py AG MI) and surface flags (-ask-only / -chat-only) support fast regression loops; the JSON enables run-to-run diffing. Three baselines to date; the latest is best-on-record: DL effectively 10/10, MI chat 3/4 (was 0/4), AG chat aggregation exact, no-answer traps honest, PR/PP procedures strong. Remaining named failures: PP-01 (acceptance bar for the next chapter), PP-03 (VOCAB-01 spell-correction), MI-04/XC-03 (heterogeneous splitter), AG-10 (Phase 1b fix_version), astro set (DESIGN-01 profiles).

10. Roadmap

10.0 Completed (Speed, Memory & Learning, Halo Connector — v9)

- **SPEED-01** — chat avg 137s -> ~24s, max 1013s -> ~68s; full 50-question eval 130+ min -> ~32 min. Levers: one fast-model front-of-pipe call replacing two 14B calls; per-question intent memoization (3 identical per-collection calls -> 1); batched vocabulary correction (2 round-trips total) and Tier 1.25 anchors (one query per word); in-process schema cache invalidated on save; spec/intent reasons capped at 8 words; [LLM TIMER] per-call instrumentation. Apple-Silicon note: concurrent predictions time-slice one GPU — cut calls and tokens, not queue positions.
- **Memory M1/M2** — chat sessions persist (resume, switch, delete); memory is an ordinary collection: capture via trigger phrases (filler-word tolerant), bare statements (whole-message interrogative veto), or the Remember-this button; facts pass a dedicated 14B resolver with a token-preservation gate before storage; notes cross-link to mentioned records (auto-confirmed at filename confidence), compose verbatim under answers question-anchored and relevance-filtered, and raise a red source-level alert NAMING the disputed field when they contradict a record. Memories tab lists notes + feedback with per-item undo.
- **Feedback learning v1 (M4a)** — thumbs on every chat answer stored with full context; a per-question feedback prior sits one rung above routing order in arbitration (settles only ties; can never overrule exact-key or grounded rungs). Verified live: a real thumbs-down flipped the next asking of the same question.
- **Halo ITSM connector (HALO-02/03)** — tickets are DATA: OAuth2 client-credentials fetcher (tenant param, /api/Status name resolution, images downloaded at sync time because their URLs carry expiring JWTs), one combined JSON per ticket as the sync/hash unit, per-item serializer (ticket header + kept human actions; payloads carry team/client/status/categories/who/action_type/datetime), the first DECLARED schema (a fact, not an inference), declared value_aliases ('resolved' -> Closed) with

injected-filter trust, and the injected-anchor retreat rule (guesses yield to grounded claims; the NA-04 law untouched). Verified: “how many tickets are resolved” answers from ticket data with the feedback prior participating.

- **RELATED-01** — high-confidence sections rejoin the ranked Related list (‘confirmed’ ranked first-class); concept sections capped at 2 with a question-relevance measurement harness (verdict: bge cannot separate junk from genuine at ~0.5 — EMBED-01 opened with the harness as its acceptance test).
- **Routing hardening** — narrow routes answering empty widen once to the top-3 centroid candidates; 2-char numeric tokens reach the anchor tiers gated by namespace-term context (‘tag 22’ anchors deterministically; ‘gain 100’ does not).
- **Remote operation** — local PostgreSQL copy runbook (pg_dump over Tailscale, repoint to localhost; 335ms RTT made even batched pipelines crawl), media_path_map for mirrored image assets, git worktrees for parallel branch work, NASAI_PORT for parallel UIs.

10.0 Completed (Routing, Arbitration & Cross-Link Delivery — v8)

- **Multi-item chat (CODE-023)** — gate/splitter/fan-out/merge; 18/18 dedicated smoke (tests/smoke_multi_item.py)
- **Reference-identifier & alias searchability** — records findable by any exact name; DL-02/DL-05 class fixed end-to-end
- **Schema inference overhaul** — LLM-primary, no junk persistence, filename constrain-don’t-hide, detail-table parent-key + discriminator corrections, schema_model routing; fleet-wide stored-vs-inferred diag (diag_schema_all.py) and git-history A/B (diag_schema_ab.py: the “good old days” scored 2/5 too — the old correctness came from deterministic post-passes, not the LLM)
- **Chat routing anchor family + Tier 1.5 centroid routing** — deterministic selection, routing-LLM call skipped on clear margins
- **Answer arbitration ladder** — wrong-collection confident answers no longer win by routing order
- **Cross-collection composition** — “compose, don’t blend”: verbatim record blocks after synthesis, identifier-prefix gated
- **Cross-links restored & delivered** — 43 wikilinks + 95 NER links, bidirectional lookups, ranked related sections, /entry full-article pages, CL-04 traversal live in answers
- **PP-02 lexical stack** — camelCase word-split, vocabulary token filtering (typos can’t veto BM25), low-coverage banner
- **Eval automation** — 50-question runner, three baselines, latency tracking (Ask ~15s avg; Chat ~48s — SPEED-01 queued)

10.0.1 Completed (Metadata SQL, Astro Schema & Evaluation — v7)

- **Metadata SQL query path (Phase 4.6)** — grounded spec extraction, parameterized templates, concept-vector field selection + filter injection, deterministic regrouping, zero-result repair, chat verbatim bypass, group-by intent

- **Structured query planner removed** — always dry-run, pure LLM latency; both modules deleted, all references removed (a dangling return during removal briefly broke all fallback queries — restored, full smoke pass)
- **BGE reranker scoped** — entity_row removed after 25-candidate demotion root-cause; KB smoke 10/10 including previously failing sFTP/PB-folder tests
- **Schema inference hardened** — cardinality signal, filename-column pre-pass → reference_identifier, entity-name/type + comments/description + numeric/date prompt rules, canonical identifier/primary_name collapse
- **Astro schema-driven (ASTRO-001/004/005/006)** — parser offset + light default, collection-level LLM schema, file_X→X coalescing, routing descriptions
- **Concept vectors CV-05** — doc_type fork deleted; tags is a ≥20%-coverage cascade step; kb_tags payload key renamed tags; CV-03b consolidation reverted (kept unwired)
- **Retrieval evaluation framework** — 50-question ground truth across 7 categories with failure buckets; drove all of the above priorities (aggregation was 1.5/10 before the metadata path; no-answer traps 4/4)
- **Mention matching removed** — cross-link discoverer now exact-identifier + name-similarity only; recon↔obsidian awaits NER

10.1b Completed (NiceGUI & Analytics — v6)

- NiceGUI front-end (Collections, Ask, Chat, Ingestion, Knowledge Graph, SQL Inspector, Debug) with shared core/ui_data.py layer and reliable inline images
- Guarded local text-to-SQL analytics engine + SQL Inspector tab + router analytics intent
- Chat tab with LLM contextualizer (follow-up detection + standalone rewrite) and history gating; faithfulness-guard NameError fixed
- Discovery/list results rendered as a table; collection lists include configured-but-not-yet-ingested collections
- Domain-specific examples removed from chat prompts (fully file-agnostic)

10.1 Completed (Ingestion Hardening — v5)

- Token-aware chunking for entity rows + doc/PDF oversized-chunk cap (P0/P0b)
- LLM schema inference: structured output, generic tags role, cross-role dedup, column pruning, prose-promotion, wide-file fallback (P1)
- Non-lossy NLP text with redundant-field dedup (P2)
- kb_tags surfaced into payload; content-priority table-type detection
- Concept vectors CV-01..04 (section_heading grouping, kb_tags IDF, TF-IDF+LLM topics, LLM cluster relabel, stale-row clearing)
- Cross-link precision CL-01/CL-02; one-hop wikilink traversal CL-04
- Default LLM switched to qwen2.5-14b-instruct-1m

10.2 Pending / Next

- Retire Streamlit (all NiceGUI tabs at parity as of v7)

- NER-based obsidian→recon links (CL-03 / CL-05) — unlocks full CL-04 payoff and is the only path to recon→obsidian links (zero exist today by design)
- Validation tab: show ALL source columns
- Background-thread cross-link trigger after ingest
- Full BBG dataset ingest (currently test subset)
- Eval failure buckets (v7): multi-item parallel queries (CODE-023, 0.5/4), cross-collection dual-source answers (1.5/6), aliases searchability (DL-05), astro synthesis detail rendering (DL-06/10), chat no-collection hallucination fallback (DL-08/PP-03), kb_docs/obsidian wrong-twin ranking
- Data Prep: Excel upload + forward-fill for merged cells (fixes NULL-identifier recon rows / CODE-024) and PostgreSQL chunk editor
- Metadata query stability (MQ-01/MQ-02): LLM spec variance and chat fan-out variance — further grounding or model routing
- Date normalization for date-typed fields (lands with ticket ingestion — enables analytics date-range counts)

10.3 Phase 4+: MCP, Chat, Advanced

- MCP as ingestion source (Halo / Confluence / Snowflake) and as query interface
- Chat memory + learning layer: persistent sessions (raw + rewritten + response), rolling summaries (keep last 3–5 turns raw), and a rated-Q&A “fast lane” (cosine $\geq \sim 0.92$) with suppression and user-approved synonym learning — built on the \$2.7 contextualizer
- Model routing (small for structured, larger for reasoning, vision for images)
- Whisper transcription pipeline; astro catalog at scale with RA/Dec cross-linking
- Deployment to Azure Virtual Desktop — fully local, no external AI, ever

11. Key Learnings & Debugging Notes

v9 additions:

- **Models classify; deterministic gates decide.** The fast 3B was caught four ways: echoing ‘recon’ as ‘recent’, leaving ‘it’ unresolved, shrinking a follow-up to a fragment, and declaring a pronoun-bearing question standalone *while citing the missing subject as its reason*. Every assertion a model makes on a hot path now crosses a tripwire (code copies text; token-subset rewrites escalate; marker words re-decide; facts pass a token-preservation gate). Same doctrine, new corollary: nothing enters permanent memory without a deterministic gate.
- **Guesses yield to claims.** Injected anchors (token=value, declared aliases) are heuristics; LLM filters that survived the groundedness guard are claims. When their combination hits zero and the claims alone do not, the anchors retreat. Honest zeros remain honest (NA-04 law).
- **Declared beats inferred when you own the shape.** The Halo serializer writes its own schema — the first collection where schema is a fact. Inference is for shapes you don’t control.

- **“Too relevant to display.”** The ≥ 0.80 related bucket fed a text-merge that had been removed — high-confidence sections displayed nowhere for weeks. When a threshold’s consumer dies, the threshold becomes a black hole; audit consumers when removing paths.
- **Small collections break big-collection assumptions.** With 11 chunks, every field clears a ≤ 20 -distinct-values bar — whole ticket bodies became “enum values” and blew a spec prompt past the model context (HTTP 400). Shape tests (value length) beat count tests.
- **Config keys can activate dormant code.** Setting `fast_model` switched THREE call sites that had been silently falling back for weeks (splitter, contextualizer, front). Grep for consumers before introducing a key.
- **Provenance lives in the text.** Memory notes carry “(User note, date)” inside `nlp_text` — every surface that renders the chunk shows its origin unconditionally; no renderer can forget it.
- **The stale-process trap is undefeated.** Restart the UI before judging any fix. Still.

v8 additions:

- **LLM for perception, math for constraints.** Every schema/spec failure traced to an unguarded LLM pick; every durable fix was grounding (real values, schema meanings, upstream intent) or a structural invariant (cardinality, composite keys, SQL algebra, leftmost-key convention). Guards are no-ops when the model is right — model upgrades need no code changes.
- **A saved wrong schema is worse than no schema** — it short-circuits every future ingest. Fail loudly, persist nothing.
- **Loose string matchers multiply.** A name-prefix anchor (`‘check’`→Checksum) routed noise in and a substring gate (`‘check’` $\subset$ `‘checksum’`) waved it through. Identifier-anchored, prefix-only matching composed cleanly. If a deterministic guard needs a third refinement round, disable it and go to the design board.
- **Repair heuristics must distinguish junk from honest zeros.** Dropping all zero-result filters turned the FIX 5.0 SP2 no-answer trap into a 200-row dump. Junk filters (schema/data confusion) are detectable; honest-but-unmatched filters ARE the answer.
- **Any text that must survive LLM synthesis cannot live in the text** — banners/flags travel beside the prose, not in it.
- **String hygiene decides groundedness** — a sentence period riding into a captured filter value (`‘= tag.’`) ungrounded a perfectly grounded answer.
- **websearch_to_tsquery ANDs terms** — one corpus-absent token (a typo) silences the entire lexical leg of RRF. Filter query tokens against the collection’s own vocabulary first.
- **camelCase names are single search tokens** — `‘OrderQty’` is invisible to a search for `‘order quantity’` until the split form is written into `nlp_text`.
- **The UI process caches code** — restart before retesting; half of all “still fails” reports were stale processes.

- **LM Studio serves chat + embeddings + schema calls on one queue** — don't test while ingesting; cap max_tokens so a runaway generation can't hold the 300s timeout.

11.1 Ingestion & Embedding

- The embedding model silently truncates past ~2,500 chars — long content must be chunked or it's invisible to retrieval. Measure the cap directly; don't trust token-usage reporting.
- What gets embedded is the ceiling on recall: a field mis-mapped to a dropped role is lost. Non-lossy text + prose-promotion protect against this.
- Entity-row chunks can share an identifier because the storage id is sequence-based — no need for _chunk_N suffixes or a separate article_id.

11.2 Schema Generation

- Structured output (response_format json_schema) guarantees JSON shape but NOT one-column-one-role — always de-duplicate across roles afterward.
- qwen-14b is reliable for schema classification with structured output; llama-3.1-8b dropped columns and over-assigned roles.
- Prioritize by signal, not by column name: prune empty columns, promote long-prose columns — keeps it file-agnostic.
- Table-type detection must check the long-description signal BEFORE the reference/enum rule, or article tables with a creator-id column get mis-routed to structured.

11.3 Cross-Linking & Concept Vectors

- Short/generic mention terms produce false positives — raise the minimum length and require meaningful context.
- Concept vectors give far better precision than substring matching; HDBSCAN finds topics without labels; multi-label assignment handles hybrid chunks.
- Rebuilds must clear stale concept vectors first, or old group values accumulate.
- LLM cluster/topic labels need collision-safe cluster ids when multiple groups map to the same label.

11.4 Chat & Analytics (v6)

- Injecting prior-turn text into the next query (bracket injection) pollutes retrieval and produces “senseless” answers on new questions. An LLM rewrite into a standalone query is far better — but it must default to standalone, or it invents follow-ups and attaches the wrong subject.
- Only feed conversation history to routing/generation on actual follow-ups; otherwise a previous topic leaks into a fresh question.
- A debug print referencing undefined variables silently disabled the faithfulness guard for months (it crashed into the except branch). Gate debug prints behind the DEBUG flag and keep them out of return-critical paths.

- Text-to-SQL is safe only behind hard guards: SELECT-only, single statement, table whitelist, forbidden-keyword block, read-only transaction with a timeout and row cap. Show the generated SQL so interpretation is auditable.
- Aggregate queries double as a data-quality lens — “count by type” immediately surfaced typo’d/duplicate values (Holdnsg vs Holdings) and concatenated cells in the source data.

11.6 Metadata Queries, Rerankers & Schemas (v7)

- Test rerankers at the production top_k — BGE gave correct order with 10 candidates and wrong order with 25 for the same query. The candidate-set size is part of the model’s behavior.
- doc_type is a weak proxy for “how should I group/handle this” — two collections can share entity_row (kb_docs, astro_catalog) and need opposite treatment. Route on the actual presence of a usable signal (tags coverage %, type diversity, folder depth).
- Raw-value embeddings pick the wrong field; concept-vector labels pick the right one — job names lexically resemble the question more than broker names do. Labels are already concept-level; concept vectors now serve routing AND metadata field selection.
- Every nondeterministic answer traced to an unordered iteration or an unguarded LLM pick — ground it (real value lists), guard it (schema-generic-only override), or repair it (zero-result filter drop), and make every tie-break sorted().
- Coalesce duplicate metadata at ingest, not query time — gain/file_gain and object/file_target made every filter a coin-flip until file_X filled empty canonical X.
- Prompt tuning has a ceiling; deterministic pre-passes don’t — three prompt attempts couldn’t stop a filename column landing in type; a 5-line regex pre-pass fixed it generically.
- Local 14B-class LLMs are unreliable at semantic label deduplication — even guarded prompts merged QA Archive – PROD Archive. That judgment class needs a larger model or an embedding-gated approach.
- Bare numbers confuse LLM wrappers — a metadata result of “947” was reinterpreted as a tag ID. Build the sentence deterministically and return it verbatim; don’t let a wrapper re-narrate SQL truth.
- An eval set beats intuition — 50 messy ground-truth questions found the aggregation hole (1.5/10), two hallucination-adjacent chat fallbacks, and the wrong-twin ranking problem that 43 passing smoke tests never touched.

11.5 psycopg2 Gotchas

- % in ILIKE ‘%term%’ → IndexError; use %% in fetchall() calls.
- JSONB returned as Python lists/dicts — don’t json.loads() again.
- pgvector centroid stored as string — pass directly, don’t round-trip through json.

12. v9 Systems: Speed, Memory & Learning, and the Halo Connector

12.1 The Front of the Pipe (SPEED-01)

One fast-model call (`front_of_pipe`, llama-3.2-3b) replaces the two serialized 14B calls that opened every turn, deciding three things at once: chat vs retrieval vs **statement** (a fact the user is telling the system), whether the message is a follow-up, and the standalone rewrite. The 3B is trusted to classify but never to assert — each of its outputs crosses a deterministic tripwire:

3B output

standalone echo

follow-up rewrite

`is_followup = false`

statement claim

fact text (memory-bound)

Downstream cuts: `llm_detect_intent` memoized per question (the three per-collection calls were byte-identical); vocabulary correction batched to two round-trips (one `unnest tsvector + one word=ANY` membership); Tier 1.25 anchors one query per word across collections; schemas cached in-process (60s TTL, invalidated on save/delete); `spec/intent` reason fields capped at 8 words (post-answer decoration, pure decode cost); [LLM TIMER] prints model, duration, and prompt size for every call and failures print the server's error body. Result: chat avg 137s -> ~24s, max 1013s -> ~68s; the full eval 130+ minutes -> ~32.

12.2 Model Roles

Role

default

`fast_model`

`schema_model`

All three are set in `config/nlp_config.json` -> `local_llm` and editable in the System Config tab (empty field = revert to default). LM Studio must have “Only Keep Last JIT Loaded Model” OFF so the 14B and 3B stay resident together; the 32B stays unloaded outside ingestion. Judgment work never moves to the small model — the splitter and contextualizer are pinned to the default model after the 3B silently broke both.

12.3 Memory (M1 + M2)

Sessions (M1). `chat_sessions` / `chat_messages` tables; every turn persists as it happens. The Chat tab resumes the most recent session, offers a session dropdown (title = first question), New chat, Delete.

Memory is a collection (M2). A remembered fact becomes an ordinary chunk in the memory collection — embeddings, BM25, vocabulary, routing, arbitration and cross-links all inherited; no parallel answer path. Provenance (“(User note,)”) lives INSIDE the text so every surface shows it. Three doors in:

- **Trigger phrases** (“remember that ...”, config memory.triggers, leading filler words tolerated);
- **Bare statements** (“the gsact file arrives at 6 AM ET”) — the front classifier proposes, deterministic vetoes dispose: any ‘?’, any interrogative word anywhere, or an auxiliary-verb lead sends the message to retrieval instead (a silent mis-capture is worse than a mis-retrieval);
- **Remember-this button** on any chat answer (stores Q + A).

Facts pass `resolve_fact` before storage: a dedicated 14B call resolves indirect subjects (“THE FILE comes at 5 AM” -> “the gsact.txt file comes at 5 AM”) under a token-preservation gate — every content token of the original must survive, the result may not be a question, else the original is stored verbatim. Born from two incidents: the generic contextualizer once rewrote a fact into the PREVIOUS QUESTION and the system memorized a question as truth.

Delivery. Notes cross-link to records they mention (existing NER scan; memory edges auto-confirm at filename confidence; `forget()` removes the note’s edges with it). Under any answer, linked notes compose verbatim as a single “**From memory:**” block — anchored to identifiers the QUESTION names (not just the winning answer), filtered by question relevance (a note rides only if it shares a content word with the question beyond the identifier; generic entity questions show all, cap 2). When a note overlaps a field LABEL of a structured record (“date column” ~ “Date Column to Check”), the UI shows a red source-level alert naming the field and the source file to update — the data stays the record; the note pressures it to stay true.

Management. The Memories tab lists every note (provenance, origin, Forget) and every feedback verdict (with undo — a stray thumbs-down changes real behavior).

12.4 Feedback Learning (M4, v1)

Thumbs on every chat answer write question, answer, collection, method and session to `answer_feedback`. Consumption v1 is deliberately conservative: a per-question net-verdict prior (normalized exact text match — feedback never transfers to questions it wasn’t given on) enters the arbitration ladder ONE rung above routing order — it settles exactly the ties previously settled by arbitrary order, and can never overrule exact-key, name, or groundedness rungs. Verified live. Planned M4b: the verified-answer cache (repeat questions answered sub-second from confirmed pairs, invalidated when their source collection re-ingests) once a feedback corpus accumulates.

12.5 The Halo ITSM Connector

Tickets are data, not documents. The fetcher (HALO/halo_fetcher.py) authenticates via OAuth2 client-credentials (tenant-aware token URL; credentials in a file OUTSIDE the repo,

config holds only the pointer), resolves status ids to NAMES via /api/Status, downloads embedded images AT SYNC TIME (their URLs carry expiring JWTs), and writes one combined JSON per ticket — the sync/hash unit, so the existing changed-file detection gives incremental sync free. --full backfills; --ticket N fetches one.

The per-item serializer emits a **header chunk** (summary + details; payload: ticket_id, team, client, status NAME, priority name, categories, opened_by, dates) and one chunk per KEPT human action (payload: who, action_type, action_datetime) — noise outcomes and system authors filtered by config, boilerplate stripped by line-prefix AND truncate-from-marker rules (disclaimers are tails, often glued mid-line). Image markers are appended to the header text so the standard renderer inlines the screenshots. Every chunk has both faces: nlp_text for retrieval, payload for SQL — “who resolved 44539” and “how was it fixed” hit the same rows.

The declared schema. The serializer saves its own schema (identifier = ticket_id, primary_name = summary, type = action_type) under the fixed stem halo_ticket — the first collection whose schema is a fact rather than an inference.

Site vocabulary. config value_aliases maps user words to data terms per collection (‘resolved’ -> Closed). Alias-injected filters are marked _injected and trusted by the groundedness guard (grounded by construction — declared, not guessed). The **injected-anchor retreat** rule: when injected anchors contradict grounded LLM claims (combined result zero, claims alone non-zero), the anchors retreat; claims are never dropped this way.

Backlog: HALO-04 learned boilerplate (paragraphs recurring across many tickets are boilerplate by frequency — replaces the config lists), HALO-05 linkable ticket ids (pure digits fail the NER distinctive filter), image dedupe by content hash, quoted-chain pruning, a Halo Sync row on the Ingestion tab + settings section in System Config.

12.6 Related Sections & EMBED-01

RELATED-01: sections scoring ≥ 0.80 used to vanish (their “merge into answer” text path had been removed — “too relevant to display”). They now rejoin the ranked, capped list; ‘confirmed’ ranks with first-class edges; image-payload merging is unchanged. Concept sections are capped at 2 and rank last; a question-relevance harness (cosine of the question vs each concept’s anchor chunk, DEBUG-logged) measured junk at 0.495–0.532 and genuine at 0.506 — **no separation**, so no floor was set. That measurement is EMBED-01’s acceptance test: a candidate embedding model must separate the broadcaster-junk from the bad-dates genuine pair before it earns the full re-embed + threshold remeasure.

12.7 Remote Operation

A month away from the NAS measured 335ms Tailscale RTT — round-trip multiplication no batching can beat. Runbook: brew postgresql@17 + pgvector, pg_dump the NAS db over Tailscale, pg_restore locally, repoint the UI to localhost:5432. The Mac copy is then the ONLY write target; the NAS copy is frozen until return (reverse dump or promote). Media referenced by absolute NAS paths renders via config media_path_map ({“/Volumes/raedsync/...”: “/Users/.../mirror/...”} — used only when the mapped file

exists; delete the key at home). Git worktrees keep branch work out of the main checkout (the FIX-UI branch lives in ../claude-fixui; NEVER switch branches in the main folder — one switch silently reverted uncommitted edits); NASAI_PORT runs a second UI alongside.

What's New in v10 — Evidence-Checked Reranking, Glossary, Halo Structuring

The Falsification Campaign

v10's defining work was a disciplined sequence of REJECTED designs, each gated by the smoke card and the 56-question eval. The reranker chose 'Charles River Environment Recovery' for "what to do if CTM is down" while ranking the correct CTM guide #2. Six mechanisms were built, tested, and five were reverted with verdicts recorded in code comments:

Variant

Caps-cap removal + title dominance

All-content-words subjects

About-df distinctiveness vetting

About-vector promotion

About-as-rerank-content (or hybrid)

BGE cross-encoder veto

Lesson (recorded): embedding similarity measures nearness, and near-but-wrong is more dangerous than far-and-wrong. The LLM reranker reads content and is the better relevance judge; deterministic overrides of it require lexical certainty, not geometry.

The BGE Veto (shipped)

After the LLM rerank + subject guard, the BGE cross-encoder scores the DEDUPED entries (title + 512 chars against the question). The veto fires only on consensus: (a) the LLM's chosen entry scores below invisible_below (0.01 — observed wrong picks: 0.001–0.004), AND (b) an entry within the LLM's OWN top-3 scores at least winner_min (0.03 — observed true winners: 0.032+). Thresholds live in config bge_veto with the calibration story. A knowledge-free scorer cannot share the LLM's wrong beliefs; the consensus condition prevents it from imposing its own (the AR-03 false winner scored 0.028 OUTSIDE the LLM's top-3).

The 'About' Infrastructure

Every doc-shaped entry carries {about, keywords, related} extracted once at ingestion with the full entry in view (background scan, resumable, on-demand button, per-collection about_scan mode in collections.json; kb_docs is 'all' because its articles arrive via the tables pipeline as entity_row). about_vectors stores embedded about lines. Though about- based

PROMOTION was falsified, the abouts serve as: audit trail of what the extractor understood; the about-closest fallback (an honest closest-match listing over anchored entries when descriptor words match nothing — ‘No exact match found’ weak-marker semantics preserved); and sectioned-entry abouts derive from the ISSUE section only.

Domain Glossary

config glossary. = {means, same_as}. Declared site vocabulary the models cannot know: CTM is the post-trade matching system, NOT Charles River (one injected line made the 14B rank the CTM guide #1 unaided); Moore is MCM; ‘bad dates’ means incorrect dates in recon files. Consumers: rerank prompt injection (only when the term appears in the question), discovery-filter synonym OR-expansion (declared phrases stay whole), and a System Config manager with journaled edits. Online enrichment (Investopedia-style) was considered and REJECTED: the glossary holds what the internet cannot know, and generic definitions feed the wrong side of the model-belief collision.

Source-Anchored Feedback

Votes now record the ANSWERING ENTRY (source_entry). A source-anchored vote counts toward a candidate only while the collection answers from that same entry — a given to the wrong document stops penalizing the collection the moment the pipeline learns to answer from the right one. Legacy votes keep collection-level effect. The verified-answer cache is unchanged (snapshot + newest- + re-ingest expiry).

Halo Structuring

Tickets are now sectioned: issue (header), responses (client-visible thread), internal_notes (hiddenfromuser), closure (the resolution outcome — sign-off, NOT the fix). The actual solution is extracted by one LLM call at serialization with the whole thread in view; it must quote the thread and name its source action ids (payload solution_sources), producing an auditable -solution chunk; has_solution=false means no chunk — no invention. Synthetic -status and -merges chunks give precise retrieval targets. Merge references (mined from ‘Other Ticket Merged’ actions + merged_into_id) and parent/child (children resolved via API at fetch time) ride in payloads, header/description text, and link edges, with family auto-follow fetching. A deterministic ticket SUMMARY table (summary word + ticket id) renders label/brief rows with click-to-expand bodies — zero query-time LLM. The Halo API panel in the Ingestion tab pulls by ticket / latest N / incremental / ALL as background tasks; the fetcher writes-if-changed (volatile-field-insensitive comparison with a changed-keys diagnostic) so unchanged tickets stop re-ingesting.

Determinism & Speed

Metadata spec cache: the LLM aggregation spec is memoized per (collection, normalized question) with a config TTL and deep-copied on store AND fetch (downstream mutates filters) — one question previously paid the 10–27s extraction up to six times across answer/discovery/ widening paths. ORDER BY was pinned on five unordered sibling/sampling queries (entry lookups now deterministically prefer the header chunk;

the metadata value sample no longer flaps the spec prompt). Arbitration title matching is token-level ('solution' no longer matches inside 'Resolution'). The discovery listing header is honest ('No direct match found — closest results:' instead of 'Found 0 item(s):' above a list). not_equals joined the filter ops with negation-aware alias injection ('not resolved' injects status != Closed). 3B front-of-pipe counters after two weeks: 516 messages, 2 escalations (0.4%), 9 statement vetoes, 1 chat-intent veto — the fast model stays.

Scale Notes (10k-ticket horizon)

Query time stays flat (capped rerank pool, centroid routing, on-demand archive)
PROVIDED pgvector gets a proper index before the corpus grows. The wall is ingestion enrichment (solution extraction + abouts per ticket); planned mitigations in order: declared extraction criteria (closed tickets only / minimum thread size), batched issue-about, fast-model option, and accepting a slow resumable background backfill. Decision deferred to a 100-ticket pilot: measure, then optimize.